BUSINESS INTELLIGENCE CENTER

BIRC 商智中心

推動校園數位人才培育

2026 後端群體驗營

時長	50 分鐘
範圍	網路·狀態碼·Port·框架

$ npm start

先從 *伺服器* 說起

同一個網址，HTML 到底是誰組出來的？

這一段要回答的問題

伺服器是什麼	一直開著的程式
CSR vs SSR	誰來組出 HTML
build 與 start	跑完就結束 vs 一直跑

伺服器，就是一個一直開著、等人來問的程式

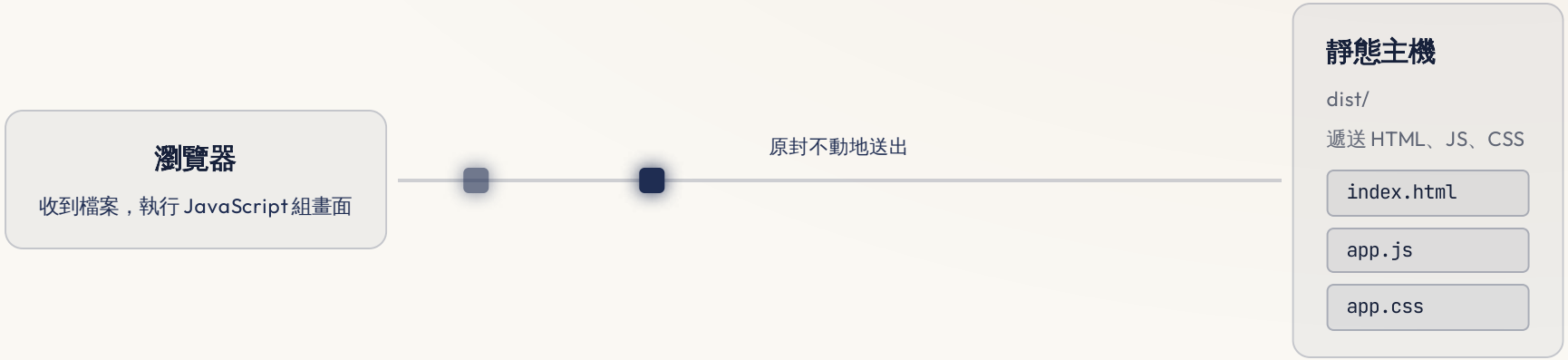
它是「程式」，不是神秘機器

一個 process，開機後掛在背景，綁定一個 port，有人敲門就回應。

它的特徵是「不會自己結束」

普通指令跑完就退出，伺服器不會。這個差別等一下是關鍵。

瀏覽器收到檔案，再由 JavaScript 組出畫面



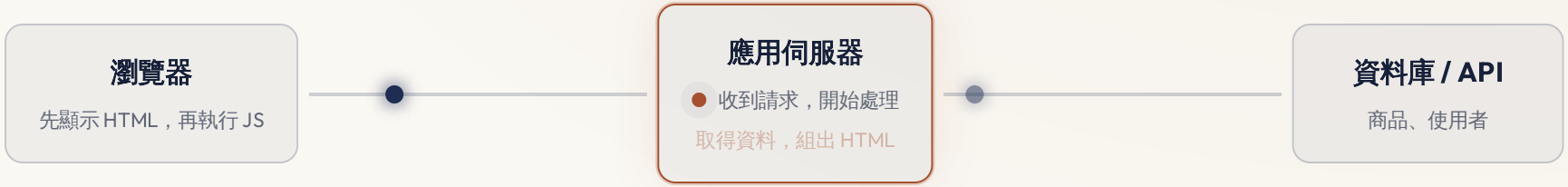
誰組出畫面？瀏覽器

靜態主機遞送 HTML、JavaScript 和 CSS，瀏覽器執行 JavaScript 後才組出主要畫面。這種渲染方式叫 CSR。

所以不需要常駐的應用伺服器

如果檔案已經在 build 時產生，Nginx、S3、GitHub Pages 這類能遞送靜態檔案的主機就夠了。

伺服器先組好 HTML，再回傳給瀏覽器



1. 瀏覽器請求 `/products`
2. 伺服器查資料，必要時等待非同步結果
3. 資料準備好後，組出並回傳 HTML，瀏覽器先顯示初始內容
4. 瀏覽器載入 JavaScript，接手按鈕、表單等互動

同一個 `/orders`，工作可以放在哪裡？

使用者打開訂單頁

`GET /orders`

CSR	靜態主機回傳程式	→	瀏覽器打 API	→ 伺服器驗證、查資料
SSR	伺服器驗證、查資料	→	伺服器先回 HTML	→ 瀏覽器再接手互動

請求在哪裡被處理

CSR 把「第一次組畫面」交給瀏覽器。SSR 把這一步提前到伺服器，但兩者都可能呼叫同一個後端 API。

為什麼有些服務需要常駐

只遞送檔案的主機可以在 build 後結束。要驗證請求、查資料、回應 API 的應用伺服器則必須持續運作。

前後端的責任邊界

瀏覽器負責呈現與互動。驗證身分、判斷能不能看訂單、查詢資料，必須由後端決定。

$ curl http://localhost:8080/api/products

Backend 101

// 網路怎麼跑 · 狀態碼 · Port · 框架的魔法

本課三條主線

看得見的請求	TCP/IP · HTTP · Port
讀得懂的回應	200 / 400 / 409 / 500
框架的真相	它替你省了什麼

你每天用的網站，拆開長這樣

在中間那兩段雙向箭頭上，發生的是

- **HTTP 協定** — 兩邊說話的規則
- **TCP/IP** — 資料怎麼在網路上跑
- **Port 號** — 找到正確的那台服務
- **狀態碼** — 這次請求的結果

> **接下來三節，把這四個詞講清楚。**

從瀏覽器到伺服器

① 輸入網址 | ② DNS 查詢 | ③ 連線 | ④ 送請求 | ⑤ 收回應

這一步在做什麼

你在瀏覽器打 `lucashsu.dev`，按 Enter

像在手機通訊錄裡輸入朋友的名字。

這一步真正傳的內容

網址列 https://lucashs

上一步 | 暫停 | 下一步 | 重播 | 步驟 1/5

DNS 負責「名字 → IP」，HTTP 負責「問什麼、要什麼」

兩件事，別搞混。

TCP/IP 四層模型

由下往上 — 線 · 路 · 包 · 話

L4	應用層	**Application** — HTTP、HTTPS、DNS、SMTP、FTP、SSH，定義「訊息長什麼樣、怎麼問怎麼答」 你寫的程式在這層·**今天上課的每一行 Java 程式碼都在這裡**
L3	傳輸層	**Transport** — TCP（可靠、慢、要握手）、UDP（不可靠、快）。`80`、`443`、`3306`、`8080` 都是這層的門牌 用 Port 號分流·**第 6 點講的 Port 號在這裡**
L2	網際網路層	**Internet** — IP 協定。負責「把封包送到全世界的正確地址」。`140.82.121.4` 就是 IP
L1	網路存取層	**Network Access** — 網路線、Wi-Fi、網卡。負責「在這條線上正確地傳位元組」

> **記憶法：從下往上 — 「線、路、包、話」**
> 網線 → IP 找到路 → TCP 找對門 → HTTP 說對話

HTTP 到底長什麼樣

請求 Request

前端 → 後端 — 第一行叫**起始行**：方法 + 路徑 + 版本

```
GET /api/products HTTP/1.1
Host: localhost:8080
Accept: application/json
```

回應 Response

後端 → 前端 — 第一行先寫版本，再寫**狀態碼** + 原因片語

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 252
```

現在就試

打開終端機貼上這行 — 不用等第 9 節

```
curl -i --http1.1 https://jsonplaceholder.typicode.com/posts/1
```

Windows PowerShell 請打 `curl.exe` — PowerShell 的 `curl` 是別名，會報錯；Mac / Linux 照打即可。

你會看到三段

`HTTP/1.1 200 OK`（狀態行）→ 一堆 Header → 空一行 → JSON 資料。和上面兩張圖對照，一模一樣。

第 9 節回來換自己的

跑起 `NativeApiServer` 後，把網址換成 `http://localhost:8080/api/products` 再打一次 — 同樣的形狀，資料變成你的。

CHECKPOINT 01

檢查點 1 — 網路與 HTTP

✓ 我能說出

- [ ] DNS 做的事
- [ ] TCP/IP 四層由下往上是什麼
- [ ] HTTP 請求有哪幾個部分
- [ ] 「無狀態」的意思

? 我還不太確定

- TCP 和 UDP 差在哪
- 為什麼要有四層，不會太多嗎
- 三次握手在幹嘛

> **馬上複習**
>
> 回到「TCP/IP 四層模型」，用「線、路、包、話」重念一次。

2xx 成功

伺服器成功處理了你的請求，而且沒有出錯

200
OK

成功，東西在這裡 — 最常見。GET 成功、POST 成功都回這個。`GET /api/products` 回的就是 200

前端怎麼處理：`response.json()` 直接用

201
Created

成功，而且「建立了東西」 — 跟 200 的差別：201 表示**伺服器新建立了一筆資源**。例如註冊新帳號、新增一筆訂單

很多實務上會混用 200 代替 201，但嚴格來說「新增」該用 201

204
No Content

成功，但「沒有東西要給你」 — 事情做完了，body 是空的。典型是 `DELETE`：刪除成功不回傳任何資料

前端陷阱：`response.json()` 會直接報錯 — body 根本沒有東西可以 parse

> **2xx 的共同點：伺服器成功處理了你的請求，而且沒有出錯。**
> 前端可以放心把資料當成有效值用。

4xx 是「你的問題」

400 **Bad Request — 你的請求格式不對** — 送出 `productId=abc`（應該是數字）。**伺服器看不懂就拒絕。**

我們的程式：`NumberFormatException → 400` ✓

401 **Unauthorized — 你「是誰」我還不知道** — 沒登入、token 過期、根本沒帶 token。**認不出你是誰**

前端：導向登入頁面

403 **Forbidden — 我知道你是誰，但你不可以** — 已登入了，但這筆訂單不是你的。**401 和 403 的差別就在這裡**

前端：顯示「沒有權限」，不要登出

404 **Not Found — 你找的東西不在這裡** — 網址打錯、路徑不存在、資源被刪了。**最常見的 4xx**

我們的程式：`GET /nope → 404` ✓

415 **Unsupported Media Type — 你的格式我不收** — 後端要 JSON，你卻送 `application/x-www-form-urlencoded`（表單格式）。**這是 body 格式錯，不是內容錯。**

我們的程式：Day 2 手拆 form；Spring 版的 `@RequestBody` 只收 JSON — **同一個 API，兩邊接受的格式不同**

401 vs 403 這組最容易混 — 面試愛問

5xx 是「我的問題」

500
Internal Server Error

我壞掉了 — 伺服器裡發生了你預期外的錯誤：程式 exception、資料庫連不上、寫炸了

我們的程式：`catch (Exception) → 500` ✓

4xx — 客戶端的問題

你傳錯了、少帶東西、想看不存在的東西。**改前端或改請求就能解決。**

5xx — 伺服器的問題

程式炸了、服務掛了。**使用者改什麼都沒用，要去修後端。**

同一份程式碼，狀態碼是這樣決定的

```
1   if（方法不對）              → 405  我不接這種請求
2   if（參數不是正整數）        → 400  你送壞了
3   if（找不到這筆商品）        → 400  你送壞了
4   if（庫存 < 購買數量）       → 409  請求合理但跟現況衝突
5   if（扣減成功）              → 200  成了
6   if（沒接住的 Exception）    → 500  我壞了
```

409 我們在 Day 2 沒講過，但它超常用

「庫存剛好被別人買完」就是最典型的 409 Conflict。

Port 號是什麼

先建立一個畫面

一棟大樓裡有幾百家公司。(find 你的名字 → IP)，接著要找到**該走哪一扇門** → Port。

IP 網址 = 大樓的地址

全網路唯一。`140.82.121.4`

由 DNS 幫你從域名查出來

Port 號 = 大樓裡的門牌

一台機器上唯一。`8080`

由程式啟動時自己指定

技術上：PORT 是 16 位元的無號整數

`0 ~ 65535`	共 65536 個
`0 ~ 1023`	「保留區」，要系統權限才能用（`80`、`443`、`22` 都在這）
`1024 ~ 49151`	「註冊 port」，要向 IANA 登記
`49152 ~ 65535`	「動態 port」，作業系統隨機指派給客戶端連線

Port 是傳輸層（TCP/UDP）的概念，不是 IP 的。

同一個 IP 可以同時跑幾十個 port，各自服務不同的程式。

你會一直遇到的 Port 號

3306	**MySQL** — 資料庫預設連線埠	`mysql -u root -P 3306`
5432	**PostgreSQL** — MySQL 的對手，開源社群常用	`psql -p 5432`
6379	**Redis** — 記憶體快取	`redis-cli -p 6379`
27017	**MongoDB** — 沒有 schema 的資料庫	`mongod --port 27017`
8080	**HTTP 替代 port** — 本課的示範 port	`http://localhost:8080`
80 / 443	**HTTP / HTTPS** — 保留區，要權限才能綁	不用打，直接省略

> **為什麼 Spring Boot 預設 8080、Express 預設 3000？**
> 因為 80/443 要管理員權限，開發時用 1024 以上的 port 最省事。

— 06 · PORT

Port 號會撞 — 到底發生什麼事

我們的 Day 2 程式裡這一行：

```
HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
//                                                  ↑ 門牌 8080
```

如果 **8080 已經被別的程式占用**（你忘了關掉昨天開的那個），**這行直接丟例外，伺服器根本開不起來**：

```
Exception in thread "main" java.net.BindException:
  Address already in use
```

（Java 版的錯誤訊息）

怎麼找出誰占了？

```
# macOS / Linux
lsof -i :8080
# 或
netstat -an | grep 8080
# Windows
netstat -ano | findstr :8080
```

解法一：關掉舊的那個

解法二：換一個 port

$ grep -rn "框架" ~/note/

什麼是框架

// 本課最需要慢慢講的一節

- 怎樣才算框架 — 先有清楚的定義
- 為什麼要有 — 從你寫過的痛講起
- 跟 Day2 差在哪 — 同一件事，兩種寫法

框架替你安排程式怎麼運作

先記住這個判斷

框架先把**啟動、組裝與執行流程**安排好。你把自己的邏輯放進指定位置，框架再於適當時機接手執行。

套件 Library

你**主動呼叫**它，決定何時使用、傳入什麼，以及如何處理結果。
例：`Math.random()`、`new Scanner()`

框架 Framework

它**主動呼叫你的程式**，依照自己的流程決定何時執行。
你要遵守它的結構與規則。例：Spring Boot、Next.js

怎麼分辨？先問：「誰掌握整個流程？」 我決定何時呼叫它 → 套件。 它決定何時執行我 → 框架。

沒有框架時（Day 2 的寫法）

`main() → new → 註冊 → start()`

你的程式掌握啟動與執行順序

有框架時（Spring Boot）

`main() → 啟動容器 → 接手流程`

你提供元件與規則，框架負責組裝和呼叫

框架幫你蓋了什麼

× 沒有框架時，程式長這樣

- `HttpServer.create(..., 8080)`
  自己建立伺服器，自己綁 port
- `server.createContext("/api/products", ...)`
  每條路由都要手動註冊
- `exchange.getRequestBody()`
  自己讀 body、解析參數
- `StringBuilder` → JSON 字串
  資料要自己轉成回應格式
- `set("Content-Type", ...)`
  header、狀態碼、回應串流都要自己寫
- `try (OutputStream out = ...)`
  資源要自己關閉

✓ 有框架時，這些都不見了

- `@RestController`
  框架負責建立伺服器與啟動流程
- `server.port=8080`
  設定寫在檔案，不散落在程式裡
- `@GetMapping("/api/products")`
  標記就是路由註冊
- `@RequestBody BuyRequest request`
  body 自動綁成物件
- `return products;`
  回傳物件，自動序列化成 JSON
- `ResponseEntity.ok(...)`
  狀態碼與 body 一起處理

> **框架沒有「消滅」這些工作，它只是把它們「藏起來」並且「一次做好」。**
> 你省下的是重複勞動，不是理解成本。

框架省了多少

× 原生版

```
// 綁 port、決定生命週期
HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
// 手動註冊每一條路由
server.createContext("/api/products", NativeApiServer::handleProducts);
server.createContext("/api/buy", NativeApiServer::handleBuy);
server.createContext("/", NativeApiServer::serveFrontend);
server.start();  // ← 忘了這行就不會跑

// 手刻 JSON：字串拼接 + 跳脫字元
StringBuilder json = new StringBuilder("[");
json.append("{\"id\":").append(p[0])
    .append(",\"name\":\"").append(jsonEscape(p[1]))
    .append("\",\"price\":").append(p[2]).append('}');
// 手動設 header 和狀態碼
exchange.getResponseHeaders().set("Content-Type", "application/json; charset=UTF-8");
exchange.sendResponseHeaders(200, content.length);
try (OutputStream out = exchange.getResponseBody()) { out.write(content); }
```

✓ 框架版

框架怎麼接手你的程式

先看三種機制，第 10 點再看它們一起運作

約定優於配置 `Conventions over Configuration`

框架先替你規定常見的放法。檔案位置、類別名稱符合規則時，**少寫設定也能得到預設行為**。

不是沒有配置，而是先使用框架的預設配置

依賴注入 `Dependency Injection`

你的類別只宣告「我需要什麼」，不負責到處 `new`。**框架建立物件，再把它傳進建構子**。

`ProductQueryController(seed)`：seed 由框架準備

註解是框架的指示牌 `Annotation`

用 `@RestController`、`@GetMapping` 標記意圖。框架讀到標記後，**替你完成註冊與分派**。

把「手動註冊」改成「宣告規則」

> **第 10 點會看到三件事同時發生**
>
> 類別放進框架會掃描的位置，框架就能找到它；標記告訴框架它的角色；需要的物件則由框架建立並傳入。

前後端如何溝通

一句話

前端發出**HTTP 請求**，後端回**HTTP 回應**。來回幾個來回就結束。**就這樣，沒有別的了。**

前端

用 `fetch()` 發請求

負責畫面

後端

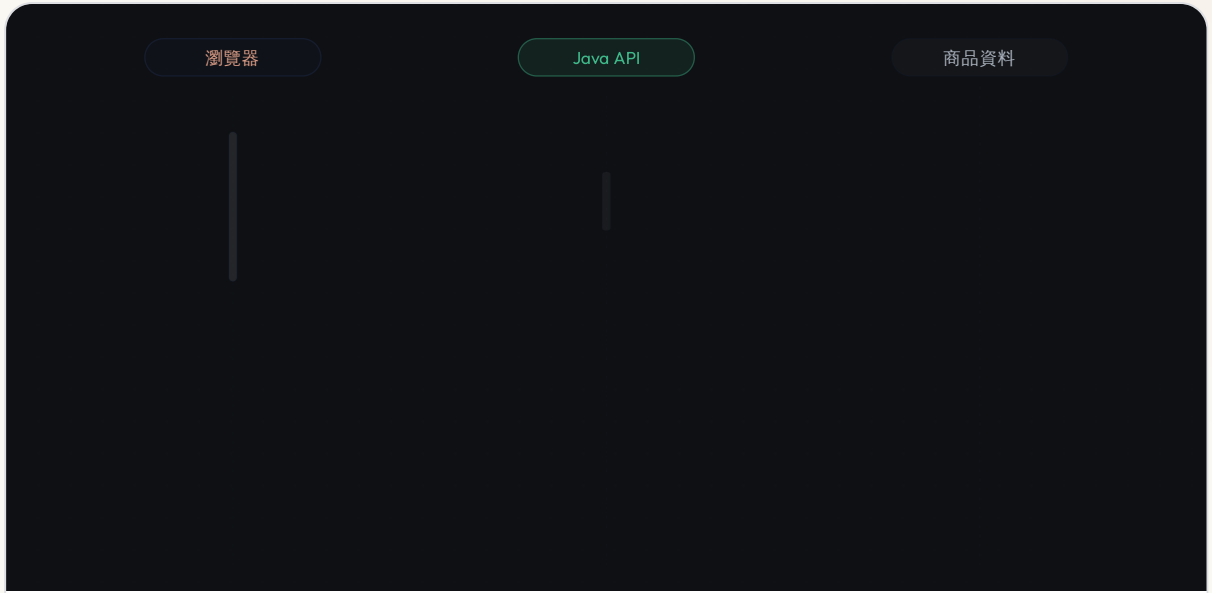
用 `HttpExchange` 收

負責邏輯

重點：前端「永遠不直接碰資料庫」。

它只知道有個網址可以呼叫，資料怎麼存它不管。

一次下單的完整對話



狀態碼是後端給前端的「明確答案」，不是錯誤訊息

把圖翻成程式碼

01 前端：發請求

```
const response = await fetch("/api/products");
//            ↑ 圖上的 GET /api/products
if (!response.ok) {
  // ok 為 false 代表狀態碼是 4xx 或 5xx
  // 200/201 才會是 true
  throw new Error("讀取失敗");
}
const products = await response.json();
```

↑回應對應圖上的「200·application/json」

02 後端：Spring Boot 回傳

```
@GetMapping("/products")
public List<Product> products() {
    return products;
}
```

↑框架把回傳值轉成 JSON，並送回 200 response

— 09 · 借鑑後端 API

前端要怎麼「借鑑」後端的 API

> **先理解一件事：API 沒有文件也能用**
> 後端寫好之後，你只要問三個問題，就知道怎麼接：

01 **網址跟方法？**
直接讀後端程式碼裡的 `createContext("/api/products", ...)` 和 `if (!"GET".equals(...))`

02 **要送什麼進去？**
看 `form.get("productId")` — key 就是參數名，`parseForm` 決定格式是 form-urlencoded

03 **會回什麼？**
看 `sendJson(ex, 200, ...)` 裡面那段字串 — **那就是你的 JSON schema**

⚡ 更快的辦法：直接用瀏覽器開發者工具

1. F12 開 DevTools　　**2.** 切到 Network 分頁　　**3.** 重新整理，看每一筆請求

Network 分頁會完整列出 **每一個 URL、狀態碼、request、response** — 這就是 API 的真相。

> **順帶一提：Day 2 的 Spring Boot 版是 JSON body，本課原生版是 form-urlencoded。**
> 格式不同，但**前端程式碼只有一行差異**（`body: data` vs `body: JSON.stringify(...)`）。這就是為什麼要先搞懂「格式」。

$ mvn spring-boot:run

配置魔法

// 同一組 API，換一個寫法

189
原生 Java

66
Spring Boot

API 完全相同

Spring Boot 怎麼啟動

01 前提：裝好 JDK 17 以上 + Maven

```
java -version    # 需要 17+
mvn -version     # 需要 3.8+
```

沒裝 Maven？從官網抓 binary 壓縮檔解壓後把 `bin` 加到 PATH。

02 啟動

```
cd springboot

# 一行，會先下載依賴再啟動
mvn spring-boot:run
```

第一次會比較慢（下載jar）。看到 `Started Backend101Application` 就是好了。

03 想打包成 jar 檔再跑？

```
mvn package
java -jar target/backend-101-1.0.0.jar
```

> **記得先關掉上一個原生版伺服器**
> 兩個都佔 `8080` → 第二個會直接 `Address already in use`。這就是第 6 點講的撞號。

— 10 · 魔法在哪

整份 Controller

ProductController.java · 66 行 — Day 2 的 BuyProductApi.java 是 189 行

```java
package com.birc.backend101;                        // 套件 → 對應到路徑，框架靠這個找

@RestController                                     // ① 「我是個 API 控制器」
@RequestMapping("/api")                             // ② 這個類別的網址都掛在 /api 底下
@CrossOrigin(origins = "*")
public class ProductController {

    private final List<Product> products = ...;  // 資料照舊是記憶體

    @GetMapping("/products")                        // ③ GET /api/products
    public List<Product> products() {
        return products;                            //    回傳物件，JSON 自動生成
    }

    @PostMapping("/buy")                            // ④ POST /api/buy
    public synchronized ResponseEntity<?> buy(
            @RequestBody BuyRequest request) {   // ⑤ body 自動綁成物件
        int id = request.productId();
        if (id < 1 || id > products.size())
```

框架幫你做了什麼

`package + 資料夾結構`	編譯器與類別載入器的慣例：**套件名必須對應資料夾**。所以 `com.birc.backend101` 必須在 `com/birc/backend101/` — Spring 靠它自動掃描。放錯就找不到。
`@RestController`	=「這個類別的方法，回傳值直接變成 HTTP 回應 body」。**取代了手動呼叫 sendJson**。它 = `@Controller` + `@ResponseBody`。
`@RequestMapping("/api")`	這一行**取代了 createContext 註冊**。整個類別的網址前綴。
`@GetMapping("/products")`	組出 `GET /api/products`。**HTTP 方法與路徑合在一個註解裡**。
`@RequestBody`	自動讀 body、解析 JSON、**塞進你的物件**。取代了 `readAllBytes()` + `parseForm()` + `getOrDefault()`，且 body 從 form 格式換成 JSON。
`ResponseEntity.ok(...)`	`ok()` = 200、`badRequest()` = 400、`ResponseEntity.status(409)` = 409。**一行搞定狀態碼 + body**。

不是 Spring Boot 永遠比較好

如果你要…	用原生 JAVA	用 SPRING BOOT
教學、做作業	✓ 看得懂每一步	× 黑箱太多
競賽限時 3 小時	✓ 零依賴不會壞	× 設定錯誤就開不起來
做真的產品	× 你會重寫到哭	✓ 別人都在用
團隊 5 人以上	× 樣板碼不一致	✓ 大家寫法統一
需要驗證/ORM/快取	× 全部自己刻	✓ 一個 annotation

> **回到第 7 點的問題：為什麼要有框架？**
> 不是因為框架比較聰明。是因為**那些麻煩事（序列化、路由、連線、JSON）每一個專案都會遇到，而且解法都一樣。**把重複的解法抽出來，就是框架。
> 框架的價值不是「少寫幾行」，是**「不用每次都重新想一次」**。

> **所以框架的前提是：你已經知道它在做什麼。**
> 今天這堂課的順序（先原生、再框架）不是巧合。

CHECKPOINT 04

全課總複習

網路

- TCP/IP 四層：線路包話
- DNS：名字 → IP
- HTTP：request / response
- 無狀態的意思

狀態碼

- 200 成功 / 201 建立了東西
- 400 請求壞了
- 401 不知道你是誰
- 403 知道但你不能
- 404 不存在
- 500 伺服器壞了

Port

- 16 位元，0~65535
- 唯一性限於同一個 IP
- 撞號 → BindException
- 3306 / 8080 / 6379

框架

- 它呼叫你，不是你呼叫它
- 控制反轉（IoC）— 說明這個現象的原則
- 約定優於配置
- 135 → 46 行

溝通

- 前端不直接碰資料庫
- response.ok 怎麼用
- 從程式碼推 API 規格

> **如果只能記一件事：框架的價值是「不用每次重新想一次」。**
> 知道它在做什麼，你才有能力判斷什麼時候該用、什麼時候該自己刻。

$ echo $?

下課

你現在可以解釋網路怎麼跑、狀態碼代表什麼、以及框架到底解決了什麼。這就是後端的真實面貌。

回家複習 `localhost:8080` 的 Port 那一頁